



Wusool

وصول

A platform connecting people to accessible places — discovered, verified, and booked in one flow.

Instructor Majd Alzahrani
Presented by Dana Alshammary

CAPSTONE PROJECT • SPRING BOOT • MYSQL • REST API



THE PROBLEM

Finding a truly accessible place is still guesswork

1

No reliable accessibility data

Maps and directories show a location's address, not whether its ramps, restrooms, or entrances actually work for someone with a disability.

2

Booking is disconnected from access needs

A user can reserve a table or a service, but has no way to confirm the venue fits their needs before showing up.

3

Business owners lack a simple way to publish it

Even owners who want to be inclusive have no structured, searchable place to declare accessibility features.

A SCENARIO

Sara just wants a cafe she can actually get into

01**Searches online**

Sara, a wheelchair user, searches “cafes in Riyadh” and gets a list with addresses and star ratings — nothing about step-free entry.

**02****Calls ahead**

She calls three cafes. One doesn't answer, one isn't sure, one says “probably fine.” She has already lost 20 minutes.

**03****Arrives anyway**

The entrance has two steps and no ramp. The trip is wasted, and she still hasn't had coffee.

The cost isn't just wasted time — it's exclusion, repeated every time she goes out.



THE SOLUTION

Wusool: verified accessibility, built into search and booking

Wusool is a Spring Boot REST API that lets businesses register places with structured accessibility details, lets users search and filter by real accessibility features, book a visit in one step, and get an AI-assisted read on whether a place fits their needs — before they ever leave home.

Same scenario, with Wusool

Sara filters cafes in Riyadh by “wheelchair ramp,” checks verified accessibility notes, asks the AI assistant one quick question, and books a table — all before leaving the house.

WHAT IT DOES

- Accessibility tagged per place (ramp, elevator, restroom, parking...)
- Search by city, category, or accessibility type
- Real-time slot booking with conflict prevention
- Gemini-powered AI assistant for accessibility questions
- Automated email + WhatsApp booking notifications



THE API

Nine resources, one consistent REST pattern

Every resource follows the same CRUD shape — GET /get, POST /add, PUT /update/{id}, DELETE /delete/{id} — so the API is predictable to consume, then layers on purpose-built endpoints for the harder questions.

User

/api/v1/user



Admin

/api/v1/admin



Business Owner

/api/v1/business-owner



Place

/api/v1/place



Place Category

/api/v1/place-category



Place Accessibility

/api/v1/place-accessibility



Place Image

/api/v1/place-image



Booking

/api/v1/booking



Review

/api/v1/review





The endpoints that needed custom JPQL

Spring Data derives simple lookups automatically, but a few product requirements — checking for a booking clash, ranking places by completed visits, matching a category name across a joined table — needed hand-written JPQL in the repository layer via `@Query`.

Booking**Booking conflicts**`isSlotTaken`**Booking****Visit history**`getUserBookingHistory`**Booking****Popularity ranking**`findMostVisitedPlaces`**Place****Cross-table search**`searchPlacesByCityAndCategoryName`**Place****Search for available places**`findAvailablePlaces`



Preventing double-booked time slots

```
boolean isSlotTaken(Integer placeId, LocalDate date, LocalTime time)
boolean isSlotTakenExcluding(Integer placeId, LocalDate date, LocalTime
time, Integer id)

select count(b) > 0 from Booking b
where b.placeId = ?1 and b.bookingDate = ?2
and b.bookingTime = ?3
```

WHY JPQL

A derived method can't express "same place + date + time"
The exclusion clause needs a hand-written condition, so
@Query was the natural fit.

EXPOSED VIA

POST**/api/v1/booking/add**

Rejects a new booking if the slot is already taken



A user's booking history, newest first

```
List<Booking> getUserBookingHistory(Integer userId)
```

```
select b from Booking b
where b.userId = ?1
order by b.bookingDate desc,
         b.bookingTime desc
```

WHY JPQL

Ordering across two columns (date, then time) together is cleaner as explicit JPQL than composing two derived-query keywords.

EXPOSED VIA

GET**/api/v1/booking/user-history/{userId}**

Returns every booking for a user, most recent first



Ranking the most-visited places

```
List<Object[]> findMostVisitedPlaces()
```

```
@Query("select b.placeId,  
count(b) from Booking b where b.status = 'COMPLETED' group by b.placeId order  
by count(b) desc")
```

WHY JPQL

An aggregate GROUP BY with a filtered COUNT has no derived-query equivalent — this is exactly what JPQL projections are for.

EXPOSED VIA

GET**/api/v1/place/most-visited**

Returns places ranked by number of completed bookings



Searching across a joined table

```
List<Place> searchPlacesByCityAndCategoryName(String city, String  
categoryName)
```

```
select p from Place p, PlaceCategory c  
where p.categoryId = c.id  
and lower(p.city) = lower(?1)  
and lower(c.name) like  
    lower(concat('%', ?2, '%'))
```

WHY JPQL

Place only stores a categoryId. Filtering by the category's name means joining PlaceCategory in the query itself — JPQL handles the implicit join and the case-insensitive partial match.

EXPOSED VIA

GET**/api/v1/place/search/{city}/{category}**

Finds places in a city whose category name partially matches



Searching for available places

```
List<Place> findAvailablePlaces(LocalTime time);  
  
@Query("select p from Place p where p.openingTime <= ?1 and p.closingTime >=  
?1")
```

WHY JPQL

Used JPQL to search for available places based on multiple conditions, such as the requested date, time, and existing bookings.

EXPOSED VIA

GET**api/v1/place/available/{time}**

Finds available locations based on a specific time determined by the user.



AI guidance and automatic notifications

1

Gemini AI assistant

POST /ai/ask-ai

Answers free-text accessibility questions (“is this place stroller-friendly?”) using GeminiService, so users get guidance beyond the tagged data fields.

2

Email confirmations

POST /api/v1/booking/{id}/send-email

EmailService sends a booking confirmation over Gmail SMTP as soon as a reservation is made.

3

WhatsApp alerts

WhatsappService (UltraMsg)

Business owners and users can be reached on WhatsApp for time-sensitive booking updates.



Thank you

Wusool — helping people reach the places built for them.